Nine Workbenches: Certified Computation in a Mathematician's Proof Language over Lean

A unified review, convergence analysis, toolchain check, and assessment of nine second-round design reports

In one paragraph. Nine agents were asked, independently, to take the architecture that a first design round had converged on for a human-scale proof language over Lean, answer twenty-two open questions left by the two syntheses of that round, and extend the language into a hybrid of proof assistant and computer algebra system built on formally verified algorithms. This document reads all nine reports in full and finds that they converge on one further idea with the same unanimity the first round showed for its architecture: a symbolic computation must return not an expression but an expression together with the exact relation it bears to its input (equality, conditional equality, equality on a domain, a witness, a complete solution set, a finite jet, or an enclosure), and the system must never promote a weaker relation to a stronger one without a theorem. Of fifty-eight design commitments tallied across the reports, forty are unanimous, and twenty-one of those are computational contracts that the first-round syntheses had not specified; the twenty-two questions receive broadly compatible answers that differ in mechanism and default rather than in direction. The review separates inherited from new convergence, tests the reports' assumptions by running the Lean toolchain rather than reading its documentation (the theorems the consensus contracts would wrap exist in the pinned Mathlib, a bounded Gröbner tactic already exists that five reports name and none ran, and a kernel-checked polynomial certificate of degree 1280 costs about a second), consolidates the nine adversarial suites into fifty-one mutations with provenance, compares itself with a parallel synthesis of the same reports and accepts that synthesis's corrections, and closes with a recommendation, questions for a third round, and per-report cards.

Prepared by Claude (Anthropic) for Vladimir Reshetnikov

5 September 2026

Status. A synthesis of design documents, plus small executed Lean experiments on the toolchain the ProveIt corpus pins. No proof language was implemented and no usability claim was measured. The tallies are this reviewer's readings of the nine texts; every mark can be checked against the cited sources, and the data files that generate the tables are shipped beside this document.

Abstract

Nine second-round reports under `docs/round-2/ideas` propose languages named Locus, Accord, Cadence, Concord, Facet, Meridian, Noema, Prism, and Vantage. Each was written with access to the two syntheses of the first round, to the same three files of the ProveIt corpus, and to the committed source of Leant, and each was asked to fold a verified computer algebra system into the proof language. This review establishes four things. First, the convergence is again very strong, but it must be read in two parts. The ten commitments the reports inherited from the syntheses are unanimous and prove nothing new. Of the forty-eight commitments outside that foundation, thirty are unanimous; six of those restate boundaries the syntheses had already fixed and three are shared worked examples, but twenty-one are computational contracts (result kinds, the execution gap, reification, dependent guards, unit versus nonzero, no canonical form) that the syntheses did not specify and that nine reports reached separately. Second, the twenty-two questions the syntheses posed receive broadly compatible answers, differing in mechanism and default, with one open spread: which definition package to build first. Third, the reports share blind spots that the first round's blind spots predicted: every report is a specification, none compiled a line of Lean, seven of nine reason from the same three ProveIt files, and although five name Mathlib's `grobner` tactic as the replacement for the retired `polyrith`, none ran it. Running the toolchain confirms the reports' documentation-based claims (native evaluation adds a per-invocation axiom; `polyrith` reports its service shut down; the semantic `Polynomial` type does not evaluate) and adds a number they lack: an unverified but kernel-checked ideal-membership certificate of degree 1280 costs about one second, so the strict checking lane is worth prototyping. Fourth, three reports independently derive the same residual-certificate theorem for finite jets of an implicitly defined series, and it is a natural early slice. The review consolidates the nine adversarial suites (182 rows) into fifty-one canonical mutations, ten of which appear in at least seven suites, proposes a first implementation ordered by how many consensus contracts each step exercises, and compares itself with a parallel synthesis of the same reports, accepting its corrections to three bridge rules, two counts, and one novelty claim.

Contents

1 Introduction

1.1 The task and the material

This is the second round of a brainstorming campaign about a better language for writing mathematics that Lean can check. In the first round nine agents diagnosed why Lean proofs are longer and less natural than a mathematician's argument and designed languages to fix that; two syntheses of those reports were then written, one by this reviewer and one in parallel by another agent. In the second round nine agents were asked to start from those syntheses and to extend the design in one specific direction: a hybrid of proof assistant and computer algebra system (CAS), in which symbolic computation is available inside the argument and is based on formally verified algorithms. The reports were also asked to answer the twenty-two questions with which the two syntheses closed. The brief itself is not in the repository; this description of it is reconstructed from what the nine reports say they were asked to do, and they agree on it.

The nine reports, their locations, and their executed companions are listed in Table 1. Each is a LaTeX article of roughly forty pages with an embedded bibliography, a proposed surface syntax, worked mathematical examples with written proofs, a Leant integration section, an evaluation plan, an adversarial test suite, and a Python program that checks the finite arithmetic of its examples with exact rational numbers. No report compiled Lean; every report says so.

Table 1: The nine second-round reports. Page counts are of the shipped PDFs; the "checks" column is the executed Python companion's own count of passed finite tests. The original directory names of three reports were changed to make the proposal names unique before this review.

Report	Subtitle	Pages	Checks	Organizing unit
Locus	Mathematical workspaces and certified computation	41	24 tests	A persistent workspace of objects, facts, routes, and policies
Accord	A certified worksheet language for reasoning and symbolic computation	37	25 tests	A mathematical transaction committing a result with its contract
Cadence	A mathematical language of certified transformations	38	267	A transformation record with a declared guarantee
Concord	A query-directed proof language with a verified symbolic core	37	459	A contracted query with a specification
Facet	Mathematical objects with certified computational presentations	37	1,100	An object with typed presentations and a relation-returning calculation
Meridian	A mathematical language with certified computation	37	27	A computed object accompanied by a theorem about it
Noema	Mathematical objects, scoped relations, and certified computation	36	25 tests	A relation-typed calculation with retained evidence
Prism	A proof language with certified mathematical computation	38	2,475	A representation anchored to an object; computations return typed claims
Vantage	Mathematical objects, explicit views, and certified computation	43	3,013	A certified change of representation for a specified observation

Three external artifacts recur in every report and are described here so that this document can be read on its own.

ProveIt is a large Lean 4 development (about 38,000 tracked Lean files at the revision the first

round audited, pinned to toolchain v4.32.0 and a fixed Mathlib), the bulk of it machine-generated combinatorics and computability, with a smaller hand-written analysis and algebra part. Both rounds draw their examples from the analysis directory: a binomial-inversion module whose kernel identity is proved by reindexing a finite sum, an autonomous-derivative module whose induction requires equality near a point rather than at it, and an Abel generating-function module whose coefficient theorem holds for any formal series satisfying a functional equation over any commutative $\mathbb{Q}$-algebra. Four second-round reports also inspect a fourth file each: an algebraic-germ module (Locus), a Lambert-function derivative module (Meridian), a floor-square-root sum (Noema), and Bell-number generating functions (Vantage).

Leant is a Haskell REPL for term synthesis and tactic suggestion over a Lean backend. It abstracts a Lean goal into a structural fragment with opaque atoms and a per-node safety bit, runs two synthesis engines (a complete one for a small logic and a heuristic one), and verifies candidates by asking Lean to elaborate them against the goal. Its committed source states that its `Verified` wrapper records acceptance by a callback rather than a kernel proof, that a negative synthesis outcome is qualified by both the engine's completeness claim and the translation's safety information, and that its tactic-suggestion path extracts `Try this` text from a successful probe without independently replaying that text. All nine reports read these files and build their provider boundary around exactly these facts.

The first round produced nine languages and two syntheses; Section 2 summarizes what they established, because the second-round reports assume it throughout.

A parallel synthesis of the same nine reports (*Mathematical Objects, Certified Computation*, by Codex) was written independently, then revised after reading a first version of this document. Section 10 compares the two, records the corrections accepted from it, and lists the ideas it contributes.

1.2 Method of this review

All nine reports were read in full, including appendices, test suites, and bibliographies. From the texts a matrix of fifty-eight design commitments was extracted (Section 4); each report's answers to the twenty-two questions were tallied (Section 5); the nine adversarial suites, 182 rows in all, were deduplicated into fifty-one canonical mutations with provenance (Section 9.2); and the worked mathematics was checked by hand where it could be (the Catalan and Abel coefficients, the Lambert polynomials, the Sturm chain, the rational root brackets, the ideal-membership counterexample over $\mathbb{Z}[X]$). All of that arithmetic is correct. One proposed contract, Vantage's telescoping rule, lacks a premise (Section 10).

Unlike the reports, this review also ran Lean. Four small experiments on the toolchain ProveIt pins (Section 7) test the claims the reports make from documentation, check that the Mathlib theorems the consensus contracts would wrap actually exist, and measure the cost of the strict checking lane that every report recommends but none priced. The experiment files and compiler logs are shipped with this document. The Python companions of the nine reports were not rerun here; the parallel synthesis reran all nine successfully, and their recorded counts are attributed on that basis.

Three data files generated by one script accompany the report: the feature matrix, the consolidated negative suite, and the question tally. Every mark in every table can be traced to a report and disputed at the margin.

1.3 Summary of findings

- **One acceptance interface, nine workflows.** Every report makes the same central move: a computation returns a value together with a proof of an exact specification, drawn from a small initial vocabulary of relations that cannot be exchanged for one another without a

theorem. The reports name the unit differently (Table 1); Section 3 shows the acceptance interface is shared, while where state lives, when computation is requested, and how authors repair a document remain open.

- **Inherited versus new convergence.** Ten of the fifty-eight commitments were fixed by the syntheses, and seven more outside that group were already stated there; the reports' unanimity on those is expected. Twenty-one unanimous commitments are computational contracts the syntheses did not specify. That is the evidence this round adds.
- **The twenty-two questions receive compatible answers,** differing in mechanism and default, with one open spread: which definition package to build first.
- **The toolchain supports the design more than the reports knew.** Every Mathlib theorem the consensus would wrap exists at the pinned revision; a bounded Gröbner tactic exists and closes an ideal-membership goal, which five reports mention and none ran; and kernel checking of a small reflective polynomial certificate costs about a second at degree 1280, which makes the strict lane worth prototyping.
- **The shared blind spots are the same as last round's.** No Lean was compiled; seven of nine reason from the same three files; performance is argued, not measured; and the "workspace" the reports design is never compared with what Lean's own sections, local instances, and `have` chains already provide.
- **A first slice suggests itself.** Three reports independently prove that a finite jet whose residual vanishes below order N agrees with the implicitly defined series below N. That theorem, a polynomial certificate checker, and guarded cancellation exercise most of the consensus contracts with no new mathematics, and the checker's arithmetic cost is now known; its soundness theorem is the work.
- **The parallel synthesis corrects this one in five places** (Section 10) and adds a bridge-registry test, a workspace-to-Lean accounting, a blind reading test, and an entailment rule for reusing certificates. All are adopted.

2 Background: what the first round established

2.1 The first-round languages and their common architecture

The nine first-round languages (Contour, Loom, MathStep, MPL, Mosaic, Motive, Outline, Reason, and Spine) shared a diagnosis: the gap between a Lean proof and a mathematician's argument is one of granularity, not foundations. A mathematician states a claim and a reason; Lean requires the representation work (casts, index bounds, side conditions, normalizations) to be spelled out, and that work is roughly a third of tactic invocations in the corpus. The languages therefore proposed a layer over Lean whose unit is a typed mathematical step with a contract, whose theorem statements are frozen before any search runs, whose omitted work becomes a scoped ledger of obligations, whose changes of representation are certified views, whose synthesis boundary never turns a failed search into a refutation, and whose accepted proofs are retained so that replay does not repeat discovery.

The parallel synthesis (*Mathematical Ideas, Checked Evidence*, by Codex) stated this as six commitments: use Lean as the host, expose mathematical assertions, retain scoped obligations, fix meaning at the search boundary, preserve accepted evidence, and evaluate against a Lean baseline given the same libraries. It added a four-way classification of edits (evidence-only, plan change, object change, statement change), the rule that a document's every asserted formal claim must be checked even if unused, a review of Leant's tactic-suggestion path, and the proposal to compare two frontends over one protocol rather than build nine languages. It closed with ten questions, listed below as S1–S10.

This reviewer's synthesis (*Nine Blueprints for a Mathematician's Proof Language over Lean*) tallied forty commitments across the nine reports (twenty-nine unanimous), audited the whole

FabiusFunction corpus rather than the twenty-three files the reports sampled (90,637 tactic-like entries in 1,004 files, 32.3% in families the reports target, a share that is flat across proof lengths), and added four points: that nearly every proposed method already exists as a Mathlib tactic, so the first deliverable is an accounting layer over existing automation; that Mathlib's `says` combinator already freezes discovered tactic text, though not proof terms; that a method's contract should be an ordinary annotated theorem type rather than a separate contract language; and that definitions, not proofs, may be the larger lever. It closed with twelve questions, listed below as B1–B12. Both syntheses were revised after reading each other, and both warn that the corpus percentages are lexical classifications, not measured savings.

2.2 The twenty-two questions

The second-round reports were asked to answer these. They are reproduced here in abbreviated form because Section 5 tallies the answers.

From *Mathematical Ideas, Checked Evidence.* S1 What is the smallest useful step contract? S2 Does a stated reason constrain the proof? S3 Which choices may automation make silently? S4 What does a verified suggestion guarantee? S5 How is infrastructure cost counted fairly? S6 How do representation views compose predictably? S7 What protects statement fidelity (what must an author inspect in a seal)? S8 What retained artifact survives replay and library upgrades? S9 How do definitions and observational replacement of witnesses enter? S10 What belongs in the published view?
From *Nine Blueprints.* B1 Is the language a document view or an input form? B2 How much of the "routine profile" is actually routine? B3 Where does the analysis plumbing belong? B4 Does an annotated theorem survive library refactoring? B5 How should a statement notice be dismissed? B6 What is the unit of replay? B7 Should every provider share Leant's three-way outcome? B8 Is structural synthesis worth an adapter at all? B9 What does a data hole look like in the published document? B10 Where does the definition layer start? B11 Who owns a method contract when two projects disagree? B12 What is the stopping rule for the whole project?

2.3 Three qualifications every report inherits

All nine second-round reports open by restating three cautions from the syntheses, in nearly the same words. The corpus audit counts source shape, not removable work, and licenses no savings estimate. An annotated theorem is the right logical interface for deductions but not the whole interface for a CAS, which also needs a representation, a denotation, an output policy, and a precision or domain contract. And Leant's safety bit, or a list of abstracted nodes, is useful metadata about a search but not a proof that the search's negative answer transfers to the original proposition. The third caution corrects a phrase in the first-round *Nine Blueprints* review that called the safety bit "the general mechanism the reports need"; the correction is accepted, and Section 9.1 records how the reports sharpen it.

3 The second-round unit: a computation that carries its meaning

3.1 Nine names for one unit

Table 1 lists what each report calls its organizing abstraction. Locus makes the *workspace* the unit: a record of environment, ordered context, object registry, fact index, selected routes, and policies, with only the first two determining what is well typed. Accord speaks of a *transaction* that commits a result only with its contract. Cadence of a *transformation record*. Concord of a *contracted query*. Facet of an object with *presentations*. Meridian of a *computed object with a*

specification. Noema of a *relation-typed calculation*. Prism of a *representation anchored to an object*. Vantage of a *certified change of representation for a specified observation*.

Beneath the names the logical content is identical and every report writes it down in the same dependent-type shape. A request q fixes an input, an output family O, and a specification Spec; an accepted answer is

$$(o, p), \qquad o : O(i), \quad p : \mathrm{Spec}(i, o),$$

checked in the exact environment and telescope of the request. Locus, Accord, Concord, Meridian, Prism, and Vantage print this structure in Lean-like notation; Cadence, Facet, and Noema write it as a judgment $E; \Gamma \vdash p : R(x, y)$. Seven of the nine then give an interface for a verified checker of the same shape, $\mathrm{check}(i, o, c) = \mathrm{true} \to \mathrm{Spec}(i, o)$. The differences are in which fields are foregrounded: Locus stresses the persistent context, Vantage and Prism the anchor-representation relation, Concord the demand the consumer states, Meridian the property lattice on results. None of these is incompatible with the others.

3.2 What the unit forbids

The reason all nine reach this unit is that they all start from the same failure of an ordinary CAS: an expression is returned without its relation to the input, and the relation is then guessed by the reader. Each report therefore tabulates the result kinds and, for each, the stronger claim it does not establish. The union of those tables is small and is reproduced as Table 6 in Section 9.1; it is an initial vocabulary that a domain package may extend with its own propositions and bridges, not a closed universe of result kinds. Every report agrees on the forbidden promotions: a witness is not a complete solution set; a product identity is not an irreducible factorization; a conditional identity is not an identity; equality on a domain is not global equality; equality at a point is not equality near it; a finite jet is not a series identity and neither is an analytic statement; an enclosure is not an equality; and non-derivability in a search calculus is not a negation.

Observation 3.1. Nine reports written from the same brief produced nine names and one acceptance interface. This is the same phenomenon the first round showed for proof steps, and it has the same explanation: the brief fixes the constraints (Lean host, verified algorithms, exact specifications) tightly enough that the acceptance condition has one natural form. The convergence is strong evidence about that condition and weak evidence about everything the reports build around it: where persistent state lives, when computation is requested, how representation routes are chosen, how authors repair a document, and which surface they type. A shared dependent pair settles what acceptance means; it does not settle those workflow choices.

4 Convergence: fifty-eight commitments

4.1 The matrix

Table 2 tallies fifty-eight design commitments across the nine reports. A filled mark means the report states the commitment explicitly and in its own design; an open mark means the report assumes or partially states it; a dash means it is absent. The rows are grouped: inherited foundation (ten rows fixed by the syntheses), result contracts, the verified CAS, representations and definitions, negative evidence and the Leant boundary, and worked material.

Table 2: Design commitments across the nine reports. • explicit; ◦ partial or implicit (a less formal statement, not absence); – absent. Columns: Locus, Accord, Cadence, Concord, Facet, Meridian, Noema, Prism, Vantage; the last column counts explicit marks. A dagger marks rows outside the foundation group that the first-round syntheses nevertheless already stated. The marks are one reviewer's coding of the texts and are shipped as `feature_matrix.csv`.

Id	Commitment	Lc	Ac	Cd	Cc	Fc	Md	Nm	Pm	Vt	#
Inherited foundation											
A1	Lean remains the checking host; no new kernel or logic	•	•	•	•	•	•	•	•	•	9
A2	Statement meaning is fixed before search; a producer cannot instantiate a meaning-bearing hole	•	•	•	•	•	•	•	•	•	9
A3	Obligations are ordered dependent telescopes with witness scope	•	•	•	•	•	•	•	•	•	9
A4	Methods are ordinary theorems with roles behind a project-owned wrapper	•	•	•	•	•	•	•	•	•	9
A5	A named reason constrains the proof; an open search reason does not	•	•	•	•	•	•	•	•	•	9
A6	Publication checks every formal assertion, including unused ones	•	•	•	•	•	•	•	•	•	9
A7	Two frontends over one typed record; a library or document view is a successful outcome	•	•	•	•	•	•	•	•	•	9
A8	Three-treatment evaluation with shared infrastructure and explicit cost accounting	•	•	•	•	•	•	•	•	•	9
A9	The corpus audit is a lexical classification, not a savings estimate	•	•	•	•	•	•	•	•	•	9
A10	Edits are classified as evidence, plan, object, or statement changes	•	•	•	•	•	•	•	•	•	9
Result contracts											
B1	A computed answer is a value together with a proof of a fixed specification	•	•	•	•	•	•	•	•	•	9
B2	An explicit taxonomy of result kinds with forbidden promotions	•	•	•	•	•	•	•	•	•	9
B3	A witness is not a complete solution set; coverage is separate evidence	•	•	•	•	•	•	•	•	•	9
B4	A conditional result is an implication; a guard is never silently added to the theorem statement	•	•	•	•	•	•	•	•	•	9
B5	An enclosure is not an equality; an interval containing zero decides no strict sign	•	•	•	•	•	•	•	•	•	9
B6	A product identity is not an irreducible factorization	•	•	•	•	◦	•	•	•	•	8
B7	Finite jet, full series identity, and analytic function are three different claims	•	•	•	•	•	•	•	•	•	9
B8	The mathematical kind of a result is separated from its validation state	◦	•	•	•	◦	•	◦	•	•	6
Verified CAS											
C1	Two lanes: a verified algorithm, and a verified checker of an untrusted certificate	•	•	•	•	•	•	•	•	•	9
C2	A third lane: proof-producing tactics, or a candidate followed by an ordinary proof	◦	◦	◦	•	•	•	◦	•	◦	4
C3	The execution gap: a correctness theorem does not certify a particular native run	•	•	•	•	•	•	•	•	•	9
C4	Native-computation axioms are audited by inventory, not by a blacklisted name	•	•	•	•	•	•	•	•	•	9
C5	Reification is part of the proof boundary; opaque atoms carry exact identities	•	•	•	•	•	•	•	•	•	9

Id	Commitment	Lc	Ac	Cd	Cc	Fc	Md	Nm	Pm	Vt	#
C6	Ideal-membership certificates with their limits stated (no negative result; powers need regularity)	●	●	●	●	●	●	●	●	●	9
C7	The `polyrith` shutdown is discussed explicitly as a design rule (service-independent evidence)	–	–	●	●	–	●	●	●	●	6
C8	A small verified core (exact scalars, polynomials, finite sums, jets), not a universal simplifier	●	●	●	●	●	●	●	●	●	9
C9	Search-free replay is not computation-free; checking cost is measured	●	●	●	●	●	●	●	●	●	9
C10	Certificate size, degree, and checker cost are bounded in the protocol	●	●	○	○	○	●	●	○	●	5
Representations and definitions											
D1	The mathematical object is a stable anchor; a representation is a certified relation to it	●	●	●	●	●	●	●	●	●	9
D2	Exact presentations are distinguished from information-losing observations	●	●	●	●	●	○	●	●	●	8
D3†	Preservation along a map is distinguished from reflection back	●	●	●	●	●	●	●	●	●	9
D4	Guards compose as an ordered dependent telescope at the retained intermediate object	●	●	●	●	●	●	●	●	●	9
D5	Coherence: routes are pinned; agreement is anchor- or observation-relative; conflicts need selection	●	●	●	●	●	●	●	●	●	9
D6	Unit is not nonzero; a CAS may not strengthen a ring to a field	●	●	●	●	●	●	●	●	●	9
D7	Guarded natural subtraction and exact division as the three-view stress test	●	●	●	●	●	●	●	●	●	9
D8	Definition packages or templates generate proof obligations, never hidden axioms	●	●	●	●	●	●	●	●	●	9
D9	A normalized-coefficient (EGF) interface as an early package	–	○	●	○	○	–	–	●	●	3
D10	A precision calculus for jets: differentiation loses one order; substitution needs a zero constant	●	○	●	●	●	○	●	●	●	7
D11	Demand-driven precision planning; precision is part of the cache key	●	–	○	●	○	○	●	●	●	5
D12	A residual certificate identifies a finite jet with an implicitly defined series	●	–	○	●	○	●	–	–	–	3
D13	Partial expressions (domain, value) as an explicit opt-in semantics beside total operations	●	○	○	○	○	●	○	○	○	2
D14	No global canonical form; CAS results never enter definitional equality	●	●	●	●	●	●	●	●	●	9
Negative evidence and Leant											
E1†	Fragment exhaustion or a safety flag is not a negation; adequacy needs a theorem	●	●	●	●	●	●	●	●	●	9
E2	Excluded middle as the example separating non-derivability from negation	–	–	–	●	●	–	–	–	●	3
E3†	The exact displayed edit is replayed; a smaller goal count is not completion	●	●	●	●	●	●	●	●	●	9
E4†	Leant stays a Haskell service behind a Lean-owned adapter with opaque handles	●	●	●	●	●	●	●	●	●	9
E5†	One budget for the whole request; cancellation; stale origins rejected; disposable workers	●	●	●	●	●	●	●	●	●	9
E6†	Shared metavariables are committed transactionally	●	●	●	○	●	●	●	●	●	8
E7†	A type is not a behavioral specification; finite tests certify only themselves	●	●	●	●	●	●	●	●	●	9

Id	Commitment	Lc	Ac	Cd	Cc	Fc	Md	Nm	Pm	Vt	#
E8	Leant is measured on compositions, separately from single-lemma lookup	•	•	•	•	•	•	•	•	•	9
Worked material											
F1	Binomial inversion kernel with additive-group generality preserved	•	•	•	•	•	•	•	•	•	9
F2	Autonomous derivatives with range-local hypotheses preserved	•	•	•	•	•	•	•	•	•	9
F3	Abel series coefficients over an arbitrary commutative rational algebra	–	•	•	•	•	–	•	•	–	6
F4	Lambert-type derivative polynomials and their recurrence	–	–	•	•	–	•	–	•	–	4
F5	Radical denesting with a branch certificate	•	•	•	•	•	•	•	•	•	9
F6	A certified rational enclosure of an isolated real root	•	–	•	○	•	○	•	•	○	5
F7	Parameter-dependent complete solving with branch coverage	–	○	•	•	○	•	–	–	•	4
F8	A ProveIt file beyond the three that the round-1 syntheses discussed	•	–	–	–	–	•	•	–	•	4

4.2 Reading the matrix

Forty of the fifty-eight rows are unanimous and forty-four are held explicitly by at least seven reports. The ten foundation rows are all unanimous, as they should be: the syntheses stated them and the brief asked the reports to build on them. Seven rows outside that group (daggered in the table) were also stated by the syntheses: preservation versus reflection, the negative-evidence boundary, exact-edit replay, the Haskell service behind a Lean adapter, whole-request budgets, transactional metavariables, and the type-versus-behaviour distinction. The reports sharpen these with new counterexamples but did not discover them. Of the remaining thirty-three rows, twenty-one are unanimous and twenty-four reach seven or more; they are the result-kind taxonomy, the two-lane CAS, the execution gap and its axiom audit, the reification boundary, ideal-membership certificates with their limits, the small verified core, the cost of replay, the anchor-and-relation discipline, dependent guard composition, pinned routes, unit versus nonzero, guarded casts, definition packages, and the exclusion of computation from definitional equality. The syntheses did not specify any of these; the reports arrived at them separately, from the same three ProveIt files and the same Lean documentation. That is the convergence this round adds. It is a count of one reviewer's marks, not a historical novelty proof, and the CSV records for every row whether the syntheses had stated it.

The spread is concentrated in six rows, and each spread is informative rather than a disagreement. C2 (a third, proof-producing lane) is explicit in four reports and implicit in the rest, which all reuse `ring`. C7 (the `polyrith` lesson) records whether a report discusses that shutdown explicitly; six do, and all nine retain service-independent evidence in any case (C9). D9 (an EGF interface) is a choice of first package. D11 and D12 (demand-driven precision and the residual certificate) are the deepest technical ideas of the round and appear in the reports that pushed the formal-series example furthest. E2 (the excluded-middle example) is a single sharp argument that three reports found. F3, F4, F6, F7, and F8 record which examples each report chose; the choice of a fourth ProveIt file (F8) separates the four reports whose examples are not shared with the others.

4.3 Ten unanimous rules beyond the first-round architecture

The following rules appear, in substance, in all nine reports. Eight were not in either synthesis; two (rules 7 and 10) sharpen boundaries the syntheses had stated, with the new counterexamples of Section 10. They are the second round's contribution, stated once.

1. A computed answer is a dependent pair of value and specification proof, checked at the request's exact telescope (B1).
2. Result kinds form a small initial vocabulary, extensible by packages, and no kind is promoted to a stronger one without a registered theorem (B2–B5, B7).
3. A verified algorithm's correctness theorem says nothing about a byte string returned by a compiled program; the strict profile connects the run to the function by kernel reduction, a reconstructed equality, or a checked certificate (C3).
4. Native computation is audited by the transitive axiom inventory, because Lean since 4.29 introduces one axiom per native evaluation rather than a single named constant (C4).
5. A reified expression must be proved to denote the original Lean term under the exact valuation; opaque atoms are identified by expression, never by printed name (C5).
6. An identity $p = \sum q_i f_i$ proves $p = 0$ from $f_i = 0$ in any commutative ring; a failed search proves nothing; an identity for p^k proves $p = 0$ only with regularity evidence (C6).
7. A representation is a certified relation to a fixed anchor; a homomorphism preserves identities but reflects them only with a separate theorem (D1, D3).
8. Guards compose as an ordered telescope instantiated at the retained intermediate object, and competing routes are resolved by pinning, selection, or a coherence theorem at the observation actually used (D4, D5).
9. "Nonzero" is not "unit"; a CAS may not silently upgrade a commutative ring to a field, an additive group to a vector space, or a formal series to an analytic function (D6, B7).
10. A negative provider outcome is not a negation unless a proof of the negated original target, or a checked certificate composed with a proved adequacy theorem, is constructed (E1).

5 The twenty-two questions, answered

Five reports answer all twenty-two questions in numbered appendices (Cadence, Concord, Facet, Noema, Vantage); Meridian and Prism answer them in tables grouped by question family; Locus and Accord answer them in the body without numbering. Table 3 gives the answer the reports share and the notable variation. The variations are compatible refinements or different defaults rather than oppositions, but several are consequential (which statuses a receipt carries, whether a witness may be delegated by specification, which coherence is automatic), so the right summary is "broadly compatible answers with differing mechanisms and defaults," not unanimity on an executable protocol. The tally is an editorial synopsis shipped as `question_tally.csv`; it is not a per-report answer matrix.

Table 3: The twenty-two round-one questions and the second-round consensus.

Id	Question	Modal answer	Variation
S1	Smallest useful step contract	An ordinary theorem behind a project-owned wrapper with roles; a richer computational interface only for algorithms, certificates, dependent outputs, or precision	Cadence and Prism name three implementation families (theorem, computation, structural plan); Locus adds a fourth request purpose (tactic edit)
S2	Does a stated reason constrain the proof?	Yes for a named method; an explicit open search does not; a different proof is a visible plan edit	Unanimous
S3	Which choices may automation make?	Proof evidence and certified representation changes of the same retained object; new witnesses, branches, motives, structures, and statements are reviewable	Prism adds delegation by specification as a third automatic class

Id	Question	Modal answer	Variation
S4	What does a verified suggestion guarantee?	The exact displayed edit replayed from its origin, with progress, root completion, and policy as separate receipts	Cadence and Meridian enumerate four statuses; Prism three; Locus and Vantage give step-by-step protocols
S5	How is infrastructure cost counted?	Share every wrapper, algorithm, and checker with the Lean baseline; report L, A_i, R_i and observed reuse	Unanimous; Facet adds a five-arm nested design
S6	How do views compose?	Ordered dependent guards at the retained intermediate object; pinned routes; coherence only at a shared anchor or registered observation	Vantage and Prism prove anchor-relative coherence; Cadence proves it per observation; none proposes global confluence
S7	What protects statement fidelity?	Render domains, quantifier dependence, structures, branches, precision; notices are distinct from obligations	Unanimous
S8	What survives replay and upgrades?	Retained terms or certificates are authoritative for the pinned environment; source is a companion; an upgrade yields a new receipt	Prism names three replay levels; Meridian three retained evidence forms
S9	How do definitions and observational replacement enter?	Specification templates with proved interface lemmas and pinned representatives; replacement only with explicit transport evidence	First template differs: formal series (Locus, Concord, Facet, Noema), partial expressions and polynomials (Meridian), EGF coordinates (Prism, Vantage), finite sums and jets (Accord, Cadence)
S10	What belongs in the published view?	Choices and substantial premises visible; routine evidence collapsed but present; rejected alternatives optional	Unanimous; Prism adds a representation view; Meridian proposes contract-surprise and guard-surprise measures
B1	Document view or input form?	Both, over one typed record; lift existing Lean first; keep only the view if authoring does not improve	Unanimous
B2	How much of the routine profile is routine?	Instrument actual attempts under fixed budgets; lexical counts cannot answer	Unanimous
B3	Where does analysis plumbing belong?	A composite ordinary theorem for locality and derivative transfer; symbolic differentiation is a separate provider	Unanimous
B4	Do annotated theorems survive refactoring?	Roles on a project-owned typed wrapper checked against the telescope; neither position nor binder name alone	Unanimous; Prism proposes a versioned wrapper schema
B5	How is a notice dismissed?	Per resolved expression or explicit project convention, keyed to meaning; dismissal never discharges a guard; a semantic change invalidates it	Unanimous
B6	What is the unit of replay?	A typed node with retained evidence in a pinned environment; certificate replay and term replay are distinct modes	Unanimous

Id	Question	Modal answer	Variation
B7	Is Leant's three-way outcome enough?	Adopt a structured taxonomy, keep abstraction diagnostics separate from proof dependencies, require checked negative evidence	Concord, Facet, and Vantage add the excluded-middle argument that non-derivability is not negation
B8	Is structural synthesis worth an adapter?	Measure compositions separately from single-lemma lookup; keep the adapter optional	Unanimous
B9	What does a data hole look like when published?	The accepted object and its full specification; alternatives are optional exploration history	Unanimous
B10	Where does the definition layer start?	Two or three templates (formal series, finite family, algebraic object, EGF), not a definition language	Same spread as S9
B11	Who owns conflicting contracts?	Namespaced, versioned packages with owners; lexical selection; never import order	Unanimous
B12	What is the stopping rule?	A library, certificate API, or document renderer is a success; new syntax must earn its place in the shared-library comparison	Unanimous

Two remarks. First, the answers to S1, B3, and B4 are identical across nine reports because they are the annotated-theorem proposal from the first round with one repair the parallel synthesis had asked for: roles are keyed to a project-owned wrapper rather than to upstream binder names, so an upstream rename breaks one adapter instead of retargeting roles across documents. Every report adopted this repair. Second, the only question with a real spread, S9 and B10, is the question the first round flagged as the larger lever; the reports agree that definitions come before syntax and disagree only on which one to package first. That disagreement is resolved by Section 9.3.

6 Where the reports differ

6.1 Genuine divergences

Partial versus total semantics. Meridian builds an explicit model of a partial expression as a pair (domain predicate, value on the domain), proves that guarded equivalences compose and specialize, and proposes it as a package. Locus uses the same pair. Noema, Prism, and Vantage keep Lean's total operations and express the same facts as conditional theorems over them, with an opt-in partial package mentioned but not developed. Both camps agree the two must not be confused (the equation $(x^2 - 1)/(x - 1) = 0$ has solution set $\{-1\}$ in the partial reading and $\{-1, 1\}$ in Lean's total reading, as Meridian and Prism both note). The difference is which is primary in the surface, and it matters for how much of the existing corpus, all written with total operations, lifts without change.

Meaning map versus relation. Facet and Vantage prefer an exact presentation to be a decoding function on valid data, because it makes the preserved object unambiguous; Accord, Cadence, and Prism prefer a relation $V(a, r)$, because it also covers lossy observations. A decoding function already handles many encodings of one object; a relation is needed only when the data is compatible with several objects. Every report uses both in practice.

First package. Section 5 recorded the four-way split on which definition package to build first. The split follows the example each report pushed hardest.

First slice. Each report names a different first end-to-end deliverable: the dyadic-germ residual jet

(Locus); annotated reindexing plus a polynomial certificate (Accord); the parameterized linear equation solved completely (Cadence); the Abel residual jet (Concord); polynomial calculation plus a guarded real-expression bridge (Facet); domain-preserving rational simplification plus complete solving (Meridian); guarded cancellation with a polynomial checker (Noema); a polynomial certificate plus the Lambert derivative step (Prism); a normalized EGF coefficient proof (Vantage). These overlap heavily; Section 9.3 orders them.

How far to trust a CAS candidate. Facet insists that "based on verified algorithms" must produce an actual verified computational core, with the certificate lane as an addition; Noema and Meridian lead with the verified reference algorithm; Accord, Cadence, Concord, Prism, and Vantage present the two lanes as equal. All agree that a certificate checker's soundness theorem must itself be proved in Lean and that the check must run under the strict policy.

6.2 Distinctive contributions

Each report contains ideas that no other report states, and several are worth keeping regardless of which design wins.

Locus. The general residual-to-jet theorem: if $F(\Delta) = 0$ for a series Δ with constant term d, the reduced polynomial $\overline{F}$ has $\overline{F}'(d)$ a unit, $J(0) = d$, and $F(J) \in (Q^N)$, then $J \equiv \Delta$ modulo Q^N; the hypothesis is a unit, not a field, and the proof is two lines once $F(J) - F(\Delta)$ is factored. Newton doubling as a second verified producer with the same acceptance interface. A valuation-sharpened multiplication precision rule. A bounded, role-indexed closure of guards in a workspace. Four request purposes (proof, data, tactic edit, computation) over one protocol.

Accord. The separation of a result's mathematical kind from its validation state (proposed, checked in session, independently replayed), which the other reports adopt in parts. A reusable two-square-root denesting theorem with its degenerate cases inside the theorem. A fifth publication check: the display's exactness, conditions, and completeness must match the certified kind. The telescoping certificate's boundary term as a source of spurious singular evaluation.

Cadence. Complete solving with set-valued output, forced by $ax = b$ at $a = b = 0$. A guarded-specialization proposition that turns "cannot evaluate at the excluded point" into a typing fact. A composition-precision rule $f(g) \equiv \tilde{f}(\tilde{g})$ modulo $X^{\min(rN,M)}$ with a proof that handles zero divisors. A dual-number algebra $\mathbb{Q}[\epsilon]/(\epsilon^2)$ as a control against field-only implementations. The observation that Mathlib's `polyrith` has lost its backend, made into a design rule.

Concord. Demand-indexed computation: the consumer states how many coefficients it needs, the planner propagates that demand backward through registered rules, and the precision is part of the cache key with restriction as a weakening rule. The residual certificate for the Abel fixed point, proved by an order-improvement argument. The status as a vector, not a green light. Three distinct negative questions (non-derivability, negation, counterexample).

Facet. The cleanest statement of exact presentation (data, validity, decode) versus observation (relation only). The example $X^2 - X$ over $\mathbf{F}_2$, a nonzero polynomial with the zero function, as the reflection failure. A five-arm nested evaluation (L_0 through L_4) that separates the accounting layer from the computational layer from the surface. Contract-directed presentation planning as a research problem with an untrusted planner and a simple route checker.

Meridian. The partial-expression model with exact and guarded equivalence and proved composition and specialization. The Lambert polynomials over $\mathbb{Z}$ with the invariant $\deg P_n = n$, leading coefficient $(-1)^n n!$, proved by induction. The example that $X \in (2X)$ in $\mathbb{Q}[X]$ but not in $\mathbb{Z}[X]$, so an injective coefficient extension still does not transport ideal membership. Two usability measures, contract surprise and guard surprise. Mathlib's `match_scalars` and `module` as existing coefficient-wise interfaces. Domain minimization with proofs as a research direction.

Noema. The counting proof of the floor-square-root sum that avoids the lossy natural-to-rational

route by supplying a balance equation in $\mathbb{N}$. A guard record (proposition, allowed dependencies, requiring operation, policy, status) and the circularity rule that a result cannot prove its own guard. The specialization $W' = W^2$, $D^n W = n! W^{n+1}$, as the minimal hybrid of symbolic derivative and local analysis. A concrete failure message rendered from the checked contract. The Lean–Mathematica interface of Lewis and Wu as precedent.

Prism. EGF coordinates as a certified representation with a proved inverse over any commutative $\mathbb{Q}$-algebra, turning coefficient convolution into binomial convolution. Delegation of a choice by specification (`construct delta satisfying ...`) as a way to keep proofs concise without a confirmation dialog. Three named replay levels. The warning that a symbolic parameter such as $(1+w)^{2n-1}$ cannot be expanded by a sparse-polynomial engine before n is known. The note that the `lean-egg` equality-saturation project now points users to `grind`.

Vantage. The three distinctions (exact presentation, restricted observation, one-way abstraction) that organize every other report's view discipline. Anchor-relative coherence proved in one line. The Bell development: normalized coefficients $n![X^n]F$ as the interface, Touchard's recurrence as polynomial algebra in $U = \exp X$, and Spivey's identity extracted by a coefficient compiler without the repeated factorial cancellation the source performs. A Sturm chain for $X^4 - 6X^2 + 1$ with its sign variations checked. "Partial information without partial proof acceptance" as the scheduler's rule under budget exhaustion.

7 Checking the consensus against the toolchain

Every report states that it consulted Lean's documentation but compiled nothing. Four small experiments were run for this review on the toolchain ProveIt pins (Lean v4.32.0, Mathlib package revision `81a5d257c8`) to test what the reports assume. The files and compiler logs are in `experiments/lean/`.

7.1 The consensus contracts exist in Mathlib

The reports' method wrappers would be ordinary theorem applications. A `#check` sweep confirms that every theorem the consensus would wrap exists at the pinned revision (Table 4). One name this reviewer guessed for the sweep, `Real.sqrt_eq_iff'`, does not exist; the theorem the reports' branch certificate needs, `Real.sqrt_eq_iff_mul_self_eq`, does. The sweep also confirms three facts the reports state from documentation: Mathlib's `Polynomial` is noncomputable (an `#eval` of its degree fails to compile), so the executable face the reports design is genuinely missing; `polyrith` now fails with the message that its external service has been shut down; and `Lean.ofReduceBool`, the constant a name-based blacklist would look for, is deprecated with the note that native evaluations are now asserted with axioms.

The sweep also ran something the reports only cite. A `grobner` tactic exists at this revision and closes `x^2 - y^2 = 0` from `x - y = 0` over $\mathbb{Q}$ without a user-supplied certificate. Five reports (Cadence, Concord, Noema, Prism, Vantage) name it, in each case as the replacement the `polyrith` documentation points to, and none exercised it. It is a Lean-core tactic wrapping the ring solver of `grind`, with default limits of 100,000 ring steps and maximum degree 1,024; a proof is proof-producing success, and failure under those limits is not a refutation. It is the natural control for the "complete decision procedure for an advertised fragment" that the reports say should stand beside the certificate checker, and its contract (coefficient fields, behaviour on failure, whether it can emit a reusable certificate) is an open item for the next round.

7.2 Native evaluation adds a per-invocation axiom

All nine reports say, from the Lean reference, that since version 4.29 native evaluation introduces computation-specific axioms, so a publication policy must inspect the transitive inventory rather

Table 4: Mathlib supply for the consensus contracts at the pinned revision, from `experiments/lean/Supply.lean`. Every entry was resolved by `#check`; the last three rows record behaviour rather than existence.

Consensus contract	Mathlib declaration(s) found	Reports that name the need
Local derivative transfer (M4)	`HasDerivAt.congr_of_eventuallyEq`, `Filter.EventuallyEq.deriv_eq`, `IsOpen.mem_nhds`, `HasDerivAt.comp`	all nine
Formal series coefficients, truncation, substitution, derivative	`PowerSeries.coeff_mul`, `PowerSeries.trunc`, `coeff_trunc`, `PowerSeries.HasSubst`, `PowerSeries.subst`, `coeff_derivative`, `PowerSeries.ext`, `X_pow_dvd_iff`	Locus, Accord, Cadence, Concord, Facet, Noema, Prism, Vantage
Formal exponential over a $\mathbb{Q}$-algebra	`PowerSeries.exp`, `coeff_exp` (coefficient is `algebraMap` of $1/n!$)	Accord, Cadence, Concord, Facet, Noema, Prism, Vantage
Principal square root and branch	`Real.sqrt_eq_iff_mul_self_eq`, `Real.sqrt_sq_eq_abs`, `Real.sqrt_nonneg`	all nine
Guarded casts (M33)	`Nat.cast_sub` (needs $m \le n$), `Nat.cast_div` (needs divisibility and a nonzero divisor), `Nat.cast_injective` for reflection	all nine
Unit versus nonzero (M6)	`IsUnit.mul_left_cancel`, `IsUnit.mul_right_inj`, `mul_right_cancel`$_0$	all nine
Finite reindexing and telescoping (M1)	`Finset.sum_bij`, `Finset.sum_nbij'`, `Finset.sum_range_sub`, `range_eq_Ico`	all nine
Real root isolation	`intermediate_value_Icc`, `StrictMonoOn.injOn`, `exists_deriv_eq_slope`	Locus, Cadence, Facet, Noema, Prism, Vantage
Ideal-membership certificate (C6)	`linear_combination` (reduces to `ring`); the bounded `grobner` tactic closes the goal directly	all nine; `grobner` named by five, run by none
Executable polynomials	the semantic `Polynomial` type is noncomputable; `#eval` of a degree fails to compile	need for an executable face confirmed
External service	`polyrith`: "no longer available, as the external service it relied on has been shut down"	confirmed for Cadence, Concord, Meridian, Noema, Prism, Vantage
Native-reduction constant	`Lean.ofReduceBool` deprecated: "assert native evaluations with axioms instead"	confirmed for all nine

than blacklist a name. On the pinned toolchain this is what happens:

```
theorem byDecide : (2 ^ 20) % 7 = 4 := by decide
theorem byNative : (2 ^ 20) % 7 = 4 := by native_decide
#print axioms byDecide   -- does not depend on any axioms
#print axioms byNative   -- [byNative._native.native_decide.ax_1_1]
```

The axiom is named after the theorem that introduced it. A blacklist of `Lean.ofReduceBool` or `Lean.trustCompiler` would pass this proof. The reports' rule is correct and is now confirmed on the toolchain the corpus uses. One nuance, found by the parallel synthesis's rerun of the same probe under a Mathlib import: there the `decide` proof depends on `propext`, so "no axioms" is a property of the bare environment. The policy distinction that matters is between the ordinary logical axioms and computation-specific ones, with the actual transitive inventory recorded.

7.3 The cost of the strict lane

Every report recommends a strict profile in which a certificate checker is a Lean function whose result is established by kernel reduction, and every report says that "search-free replay is not computation-free" and that the cost must be measured. None measured it. The experiment in `PolyCert.lean` defines dense integer polynomials as lists, an unverified ideal-membership checker `check p fs qs` that tests whether $p - \sum q_i f_i$ is the zero polynomial, and the certificate family $X^n - 1 = (1 + X + \cdots + X^{n-1})(X - 1)$. Table 5 gives the cost of `decide` (elaboration plus kernel type checking), of `native_decide`, and, from `LinComb.lean`, of the proof-producing route through Mathlib's `linear_combination` on the same identities.

Table 5: Cost of three routes for the certificate $X^n - 1 = (1 + X + \cdots + X^{n-1})(X - 1)$ on the pinned toolchain. Kernel and native columns give the profiler's tactic-execution plus kernel type-checking entries for two runs on 6 September 2026 (earlier runs varied by a factor of two to three with machine load; the shape is stable). The `linear_combination` column is the `ring` time plus the kernel check of the resulting term; its sizes differ because the file was generated separately (there is no 640 or 1280 case), and it excludes the roughly fifty seconds of importing Mathlib. Axiom inventories: `decide`, none in this import-free file; `linear_combination`, the standard `propext`, `Classical.choice`, `Quot.sound`; `native_decide`, one extra per-invocation axiom.

Degree n	Kernel `decide` (run 1 / run 2)	`native_decide` (run 1 / run 2)	`linear_combination`
10	33 ms / 24 ms	1.29 s / 1.42 s	0.27 s ($n = 10$)
40	68 ms / 72 ms	1.31 s / 1.44 s	0.35 s ($n = 40$)
160	0.25 s / 0.25 s	1.32 s / 1.44 s	0.73 s + 0.14 s kernel ($n = 160$)
640	0.75 s / 0.83 s	1.33 s / 1.44 s	2.01 s + 1.65 s kernel ($n = 320$)
1280	1.29 s / 1.41 s	1.33 s / 1.45 s	—

Three things follow, each with its scope stated. In this family the strict lane grows roughly linearly in the degree and stays near a second at degree 1280; that is because the second factor has two coefficients, so the multiplication is not dense-by-dense, and no quadratic or general scaling claim is supported. Native evaluation shows a size-independent cost of about 1.3 s per theorem in this environment (the profiler attributes it to the type-checking phase of the native step, so it is not established to be compilation), so at these sizes it is slower than the kernel and buys an axiom for nothing; the crossover exists but has not been located. The proof-producing route through `ring` is competitive at small sizes and grows faster, and its kernel check of the produced term at $n = 320$ already exceeds the whole reflective check at $n = 1280$. The corrupted certificate (one coefficient changed) is proved rejected by both routes: `corrupted_rejected_kernel` states that the checker returns `false` and is accepted by `decide` with no axioms.

These numbers are for an unverified checker; the missing pieces are its denotation into `Polynomial`, its soundness theorem, and the reification of a Lean goal into its input, which is implementation work of the kind the reports specify. The checker also silently truncates unequal certificate lists (a protocol must reject an arity mismatch), and the family's generator denotes $X^n - 1$ only for $n \geq 1$. The numbers are for dense lists of small integers; sparse multivariate polynomials with large rational coefficients will behave differently, and kernel arithmetic on `Int` is accelerated only through `Nat`. What they settle is narrower than the reports hoped and still useful: a small closed coefficient-list certificate can be checked strictly at modest cost, so a prototype of the strict lane is worth building, and the point at which `native_decide` earns its axiom is something to measure rather than assume.

7.4 The residual certificate and the precision loss, checked

The same file checks the two small facts three reports build their formal-series design on. The Catalan jet $c = 1 + X + 2X^2 + 5X^3 + 14X^4 + 42X^5$ has residual $c - 1 - Xc^2$ with zero coefficients below degree 6 and coefficient -132 at degree 6, so `decide` accepts `catalan_jet_ok` and `catalan_not_exact` in a few milliseconds and with no axioms; this is MERIDIAN's example and the concrete form of LOCUS's and CONCORD's residual theorems. And the derivatives of X^5 and 0, which agree below order 5, do not agree below order 5 after differentiation, which is the precision-loss rule of D10 (`deriv_loses_order`). Neither fact needed checking; the point is that the arithmetic half of the reflective machinery the reports describe is a few dozen lines of plain Lean. The other half, the theorem connecting an arbitrary exact solution in $R[[X]]$ to its jet and the interpretation of the coefficient list, is not in this file and is the work.

8 Assessment

8.1 What is strongest

The result-kind discipline. The single most valuable outcome of the round is a small initial vocabulary of result relations with explicitly forbidden promotions, reached independently nine times and extensible by packages. It is implementable as ordinary Lean propositions, it makes the classic CAS errors (the cancelled pole, the wrong branch, the missing root, the analytic reading of a formal identity) into type errors at the request boundary, and it is what distinguishes the proposal from "call a CAS and hope `ring` can close the gap." Section 9.1 consolidates it.

The execution gap. The reports' insistence that a verified algorithm's theorem does not certify a native run, and that the strict profile must connect the two by kernel reduction or a checked certificate, is correct, is now confirmed against the toolchain, and is priced (Section 7.3).

Preservation versus reflection. The distinction between transporting an identity along a homomorphism and reflecting one back is stated in every report, illustrated with sharp examples ($\mathbb{Z} \to \mathbb{Z}/2$, $X^2 - X$ over $\mathbf{F}_2$, $X \in (2X)$ in $\mathbb{Q}[X]$ but not $\mathbb{Z}[X]$), and is the correct answer to the first round's coherence question: routes compose forward without global confluence, and reflection is always a separate theorem.

The precision calculus. The rules for finite jets (addition and multiplication at min, differentiation at one order less, zero-constant substitution, unit inversion, CADENCE's $\min(rN, M)$ for composition, LOCUS's valuation sharpening) are proved in the reports, agree with one another, and are exactly what `PowerSeries.trunc` and `coeff_derivative` would wrap.

The residual certificate. Three reports (LOCUS, CONCORD, MERIDIAN) derive that a jet whose residual vanishes below N agrees below N with the implicitly defined series, and two of them notice that this identifies a computed object with an *arbitrary* solution of the functional equation without replacing it by a canonical construction. It is a reusable theorem, its hypothesis is a unit rather than a field, and its checker is a polynomial evaluation.

8.2 What is weak, over-engineered, or under-specified

Specification without a prototype. All nine reports are design documents. Each says a first slice should be built before the grammar grows, and each stops short of building it, although Section 7.3 shows that the checker half of the first slice is trivial to prototype. The Python companions check arithmetic, not designs; a report that had spent its companion effort on two hundred lines of Lean would have learned more.

The workspace is never compared with Lean. LOCUS designs a persistent workspace with a registry, a fact index with role-indexed guard closure, and route selection; ACCORD, MERIDIAN, and PRISM design similar records. None asks what Lean's `section`, `variable`, local instances,

`have` chains, and `simp` sets already provide, nor which parts of the workspace are an accounting layer over those. The first round's finding, that nearly every proposed method already exists and the missing piece is accounting, applies again: the supply table shows that the theorems exist; the reports' contribution is the retained record and the contracts, and they should say so. The parallel synthesis supplies the allocation this review asked for (Section 10).

Same three files, again. Seven of nine reports reason from the binomial-inversion, autonomous-derivative, and Abel-series files, because both syntheses discussed them. The four reports that opened a fourth file (Locus, Meridian, Noema, Vantage) produced the round's most distinctive examples: the algebraic germ, the Lambert polynomials, the counting proof, and the Bell coefficient compiler. The brief did not require the three files; the reports chose them by inertia.

Cost is asserted. Every report says checking must be measured and none measures it. Every report says `native_decide` must be disclosed and none notices that it costs more than the kernel at the relevant sizes.

Some machinery is premature. Locus's bounded rewrite graph with guard-indexed edges, Accord's equality-saturation engine, Facet's contract-directed planner, and Meridian's domain minimizer are all correctly labelled as later work, but they consume pages that would have been better spent on the soundness theorem of one checker.

8.3 Blind spots common to all nine

- No report ran `grobner`, although five name it as the `polyrith` replacement; none states its limits or its contract.
- No report discusses how a reflective checker is proved sound in practice: the `Decidable` instance, the denotation of a list of coefficients, the evaluation homomorphism, and the cost of kernel reduction through `Int` arithmetic. These determine whether the strict lane is usable.
- No report addresses multivariate polynomials beyond naming them, although the ideal-membership certificate is multivariate in every real use.
- Prism and Noema say that evidence is authoritative only for its pinned environment, but no report considers a toolchain upgrade that changes the native-evaluation axiom scheme itself, which happened at 4.29 and will happen again; an allowlist whose names are renamed is not a revalidation.
- No report treats collaboration beyond package ownership; merging two authors' edits to one workspace, which Accord raises, is answered by "semantic review."
- Every report's Leant section reproduces the syntheses' static findings. Noema and others rightly ask that compositions be measured separately from single-lemma lookup, but no report names the population of obligations on which to measure: those generated by guarded cancellation, complete solving, and residual-jet assembly.

8.4 What the convergence does and does not show

The nine reports were written from the same two syntheses, the same three files, the same Leant sources, and the same Lean documentation, by models of one family. Agreement on the inherited rows is therefore expected and shows nothing. Agreement on the twenty-one unanimous computational rows shows that the constraints of the brief determine the acceptance condition of the computational boundary: given Lean, verified algorithms, and exact specifications, a value with a proof of a fixed relation at the original target is the sensible interface, and nine attempts found it. It does not show that the interface is usable, that the workspace reduces work, that any of the nine surfaces is better than annotated Lean, or which representations, certificate formats, execution policies, guard interfaces, and defaults to choose; a shared acceptance condition leaves all of those open. They remain the three-treatment experiment's job, and all nine reports say so.

9 Additions

9.1 The promotion table

The nine result-kind tables differ in wording and coverage. Table 6 is their union, arranged as examples of the additional evidence a particular inference needs. It is the artifact the reports describe but none quite writes down: every row is a pair of proposition families, every bridge is an ordinary theorem whose hypotheses must all be supplied, and a request that names the stronger claim cannot be satisfied by the weaker one. It is not a total ladder and not a closed inventory; a package may add rows. Three rows were repaired after the parallel synthesis showed that a premise was missing (Section 10), which suggests the registry's own test: for each bridge, omit one premise and demand that the promotion fail even when the underlying computation is correct.

9.2 The consolidated negative suite

The nine adversarial suites contain 182 rows. Mapped onto canonical mutations they yield fifty-one distinct families and 183 report-to-family incidences (the map is many-to-many: five source rows fall in two families, and four families merge two rows of one report). Ten families appear in at least seven suites and seven more in five or six; these seventeen are a reasonable frequency-selected core for a first implementation, with the caveat that frequency measures how often a table happened to include an instance, not how serious the mutation is. Table 7 lists them; Appendix B lists all fifty-one with provenance, and the CSV gives the per-report identifiers so that each canonical row can be traced to the original tables.

Three observations. First, the fifteen families that only one report contains (M10 the wrong formal root branch, M20 search instantiating a statement hole, M26 a new constructor, M28 a finite test for a universal claim, all from Locus; M37 a noncommutative input and M50 algebraic-number equality from a shared polynomial, from Accord; M40, M41, M44, M51 from Meridian; M42 and M49 from Prism; M43 the cyclic guard from Noema; M47 from Facet; M48 from Concord) are not weaker than the popular rows; several protect statement integrity more directly. A consolidated suite should keep all fifty-one. Second, several canonical rows are umbrellas whose subcases need different theorems and must stay separate as executable tests: M6 covers cancelling a regular element, inverting a unit, and eliminating a power, and the integer 2 is regular but not a unit, so a checker that accepted regularity for an inverse request would be wrong; M27 covers a timeout, a malformed status, and Facet's complete non-derivability of excluded middle, which is a semantic counterexample rather than a budget failure; M34 covers three certificate formats with three soundness theorems; and M8 as Meridian states it changes a solution set, not merely a domain. Third, every report insists that each mutation ship with a positive neighbour so that a system that rejects everything cannot pass; the CSV records the mutations in prose, without executable inputs, expected propositions, or positive partners, so it is a backlog for a test register, not a suite a prototype can "pass."

9.3 A first slice ordered by contract coverage

The reports name nine different first deliverables. Ranking each candidate by how many of the unanimous contracts it exercises, and by how much of it the supply table shows already exists, produces one order.

1. **A polynomial identity certificate with a proved checker.** Exercises B1, C1, C3, C4, C5, C6, C9, and the execution gap; the checker's arithmetic is a few dozen lines of Lean and its cost is now known (Section 7.3); the soundness theorem, the evaluation homomorphism, and the reification bridge to a Lean expression are the work. `linear_combination` and the bounded `grobner` are the controls, with their contracts documented. Named as a first slice by Accord, Facet, Noema, Prism, and, with guarded cancellation, by the parallel synthesis.

Table 6: Result kinds and the bridges between them. A bridge is an ordinary theorem whose hypotheses must be supplied; no bridge is automatic. Reports in the last column state the row explicitly.

Weaker claim	Bridge to the stronger claim	Stronger claim	Stated by
Non-derivability of an abstraction A in a search calculus	Never by itself. A checked proof of $\neg A$ together with a proved $P \to A$ gives $\neg P$; absence of a derivation of A is not a proof of $\neg A$	Negation $\neg P$	all nine
Candidate set S	Soundness $\forall x \in S,\ P(x)$ *and* coverage $\forall x,\ P(x) \to x \in S$; a witness plus coverage is not enough ($P(x)$ is $x = 0$, $S = \{0, 1\}$)	Complete solution set S	all nine
Product identity $p = c \prod f_i^{e_i}$	Irreducibility of each f_i and a normalization convention	Irreducible factorization	all but FACET
Conditional identity $G \to e = e'$	A proof of G, or a case split whose cover implies the disjunction of guards	Identity $e = e'$	all nine
Equality on a domain D	Agreement on the complement, or an extension theorem for a new object	Global equality	all nine
Equality on an open set $U \ni x$	Openness of U turns the identity on U into eventual equality at x; equality at x alone never suffices ($f(t) = t$, $g(t) = 0$)	Equality near the point (germ); derivative transfer	all nine
Jet: coefficients agree below N	A uniform theorem for every N, or a recurrence with initial data and uniqueness	Series identity	all nine
Series identity	Convergence and an evaluation or realization theorem on a stated domain	Analytic identity	all nine
Enclosure $l \le x \le u$	Collapsed bounds $l = u$, or a limiting theorem over a family of enclosures; a nondegenerate enclosure gives only bounds, and a strict sign only when $0 \notin [l, u]$	Equality	all nine
Residual of a candidate J vanishes below N	An existing root Δ of the same equation, the same constant term (branch) $J(0) = \Delta(0)$, and a unit reduced derivative there (or order improvement of the fixed-point map); over $\mathbb{Z}/4\mathbb{Z}$ the nonunit derivative 2 admits $J = 2t$ with zero residual and $J \not\equiv 0$	Jet of the implicitly defined series	LOCUS, CONCORD, MERIDIAN
Identity after a homomorphism	Injectivity or another reflection theorem for that map	Identity before it	all nine
Ideal membership over $\mathbb{Q}$	None in general ($X \in (2X)$ in $\mathbb{Q}[X]$, not in $\mathbb{Z}[X]$)	Ideal membership over $\mathbb{Z}$	MERIDIAN
Finite behavioral tests pass	A proof of the universal specification	Universal behaviour	all nine

Table 7: The seventeen mutations that at least five of the nine adversarial suites contain. The count is of suites, not of text mentions; several more reports discuss each in prose.

Id	Mutation	Required behaviour	Suites
M4	Replace neighbourhood equality by equality at one point before differentiating	Derivative transfer stays open naming the missing eventual equality; no global smoothness is added	9
M15	Present verified roots as the complete solution set, or omit a root	Soundness may pass; completeness or coverage must fail	9
M1	Omit the inclusive endpoint of a finite reindexing	Coverage fails at the endpoint; the advertised method is not silently replaced by another proof	8
M7	Cancel a denominator that can vanish, or evaluate a cancelled expression at the excluded point	Keep the guard, split cases, or return a piecewise result; never publish the unconditional equality	8
M9	Return the negative radical candidate with the correct square	Reject the principal-root specification despite identical squares	8
M17	Deliver a valid result after undo or an origin change	Mark it stale; it cannot mutate the current state	8
M21	Cache or display suggested tactic text that differs from the tactic that was probed	Replay the exact displayed edit from the same origin before calling it verified	8
M25	Insert a false but unused intermediate assertion	Reject the document even if the final theorem has another proof	8
M27	Present search exhaustion, a timeout, or a missing status field as a refutation	Return a non-logical outcome unless a negation is checked at the original target	8
M12	Differentiate a jet and keep its input precision	Lower the output order or request one more input coefficient	7
M13	Read a formal identity or jet as an analytic statement	Require a convergence, realization, or remainder theorem	6
M16	Reuse a branch-local fact in a sibling branch, possibly through a cache	Reject the context embedding regardless of printed similarity	6
M6	Cancel a nonzero non-unit, or use a power certificate, in a ring with zero divisors	Require unit or regularity evidence; do not strengthen the ring	5
M14	Promote finitely many agreeing coefficients to a series identity or a universal formula	Retain only the finite statement; the universal claim stays open	5
M23	Admit an unapproved axiom transitively, or forge an evidence tag	Fail the publication policy even if the term typechecks	5
M33	Transport natural subtraction or natural division to a ring or field without its guard	Expose the order or divisibility obligation	5
M38	Display an enclosure as an exact decimal, or infer a sign from an interval containing zero	The certificate stores exact bounds; a zero-containing interval is inconclusive	5

2. **Guarded rational cancellation and complete solving of $ax = b$.** Exercises B3, B4, B5, D6, D13, and the partial-versus-total distinction; uses `mul_right_cancel`$_0$ and `IsUnit` lemmas; is the shortest example whose complete answer is set-valued. Named by Cadence, Meridian, Noema.
3. **The residual-jet certificate.** Exercises B7, D8, D10, D11, D12, and the arbitrary-solution discipline of the Abel theorem; wraps `PowerSeries.trunc`, `coeff_derivative`, and `HasSubst`; the Catalan instance's arithmetic is checked in Section 7. Named by Locus and Concord; the general theorem is Locus's, and its $\mathbb{Z}/4\mathbb{Z}$ negative neighbour is the parallel synthesis's.
4. **Two theorem adapters: finite reindexing and local derivative transfer.** Exercises A4, A5, B4 (M1 and M4); wraps `Finset.sum_bij` and `HasDerivAt.congr_of_eventuallyEq`; is

the accounting layer over existing automation. Named by ACCORD, FACET, NOEMA.
5. **The EGF coordinate package.** Exercises D2, D3, D8, D9 and the definition-first thesis; VANTAGE's Bell development is the held-out test. Named by PRISM, VANTAGE.
6. **The Leant adapter with exact-edit replay.** Exercises E3–E8; last, because it depends on the typed request boundary the earlier steps fix.

Each step's negative neighbours are in Table 7. The first three steps need no new mathematics and no surface syntax; they are Lean library work with retained records, and they would already decide whether the strict lane's cost and the checker's soundness proof are as tractable as the numbers suggest.

9.4 What the strict lane needs that the reports do not say

The reflective checker in Section 7.3 is fast because the kernel reduces `List Int` arithmetic with accelerated `Nat` operations. Three design consequences follow for the verified core the reports specify.

- **Choose checker data for kernel reduction, not for elegance.** Lists of integers reduce; the semantic `Polynomial` type, built on `Finsupp` in a noncomputable section, is not meant to. Mathlib's own reflective tactics already keep a computational representation beside the semantic one; the reports' "executable face" is that pattern with a retained record, and its denotation theorem into `Polynomial` or `PowerSeries` is the reification bridge of C5.
- **Prefer `decide` to `native_decide` until measured otherwise.** At every size in the tested family the kernel was faster and added no computation axiom. The accelerated profile the reports permit should be enabled by a measured threshold on the actual workload, not by default.
- **Make the checker's soundness theorem the first verified artifact.** The whole architecture reduces to $\mathrm{check}(i, o, c) = \mathrm{true} \to \mathrm{Spec}(i, o)$ for a handful of checkers. Proving one such theorem end to end, including the denotation of the reified input, is a better test of the design than any grammar.

9.5 A fourth-file rule

The most original material in the round came from the four reports that read a ProveIt file no synthesis had discussed. A third round, if there is one, should require each report to develop one example from a file that no previous report or synthesis has cited, and should treat the shared three files as controls. The corpus has a thousand files in the analysis directory alone; the design's claim to generality is only as strong as the variety of examples it survives. The rule is for development material, not for evaluation: a file becomes development material the moment it is inspected, so the held-out task of the eventual comparison must be sealed separately, and the Bell and Abel files, around which the reports designed, cannot serve as held-out tests. An uncited filename does not by itself prevent near-duplicate tasks or dependency leakage; VANTAGE's exclusion rule for aliases and downstream consequences still applies.

10 The parallel synthesis

A second synthesis of the same nine reports, *Mathematical Objects, Certified Computation* (Codex, `docs/round-2/synthesis/`), was written independently and then revised after a critical review of a first version of this document; three review memos on this document's design claims, tables, and toolchain experiments accompany it. This section records the agreement, the corrections accepted here, what that synthesis adds, and where the two still differ.

10.1 Agreement

The two syntheses reach the same conclusions by different routes. It tallies eight broad commitments across the nine reports with 241 source references and finds all eight explicitly supported by every report: relation-specific results, fixed objects with certified presentations or observations, scoped guards, a justified execution and reification boundary, theorem interfaces separate from search policy, exact-target providers, retained publication evidence, and a fair comparison. This review's fifty-eight finer rows refine those eight; the two matrices have different denominators and answer different questions, and neither should be turned into a score. Both syntheses judge the same distinctive ideas worth keeping (Locus's residual theorem, Vantage's three representation relationships, Meridian's and Prism's symbolic calculus with a symbolic index, Noema's guard record, Concord's arbitrary-solution discipline, Accord's certificate architecture), both recommend one typed request-and-evidence protocol exercised first by guarded polynomial and rational computation and one residual problem, both insist the checker's soundness theorem and reification bridge precede any grammar, and both accept a library-and-linter outcome. It reproduced all nine Python companions, which this review did not.

10.2 Corrections accepted

- **`grobner` was noticed.** A first version of this document said no report mentioned it. Five do (Cadence, Concord, Noema, Prism, Vantage), as the replacement the `polyrith` documentation names; none ran it. It is also a bounded tactic, not a complete decision procedure. Sections 1, 7, 8, and 12 were corrected.
- **Three bridge rules lacked a premise.** A witness plus coverage does not make a complete solution set without soundness; equality at a point plus openness does not give equality near the point; and "never" was wrong for enclosure-to-equality, since collapsed bounds $l = u$ do give $x = l$. Table 6 now states the full hypotheses, adds the $\mathbb{Z}/4\mathbb{Z}$ nonunit counterexample to the residual row, and restates the non-derivability row so that only a checked proof of the abstraction's negation transfers.
- **Two counts were wrong.** The nine suites contain 182 rows, not 183; 183 is the number of report-to-family incidences. And fifteen families are singletons, not four; two (M49, M50) had been omitted from the list.
- **"Thirty new commitments" overstated the novelty.** Seven rows outside the foundation group were already stated by the first-round syntheses; they are daggered in Table 2, and the count of contracts the syntheses did not specify is twenty-one. The matrix is one reviewer's coding, and its "new" group was a partition label, not a historical comparison, until this revision added the flag.
- **The benchmark's scope was overstated.** The tested family multiplies by a fixed two-term factor, so no scaling law follows; the native cost is a profiler entry, not an established compilation overhead; the shipped file had lost its corrupted-certificate case, which is now a theorem; and the Abel instance was never in the file. Section 7.3 was rewritten with fresh runs.
- **Smaller wording repairs.** A guard is "never silently added to the theorem statement" rather than "never promoted to a hypothesis," since a visible branch may assume one; an interval containing zero decides no strict sign rather than nothing; the result-kind family is an initial vocabulary rather than a closed one; the nine units share an acceptance interface rather than being mere spellings; the questions receive compatible answers rather than the same one; and the row on `polyrith` now counts what it counts.

10.3 What it adds, and is adopted here

- **A test for the bridge registry itself.** For each implication between guarantees, omit one premise and require that the promotion fail although the underlying computation is correct.

This tests the specification, not the checker, and is added to the suite as three cases (the repaired rows) plus the cyclic-guard case M43, which it accepted from this review.

- **A nonunit counterexample for the residual theorem.** Over $\mathbb{Z}/4\mathbb{Z}$ with $F(Y) = 2Y$, $\Delta = 0$, and $J = 2t$: zero residual, derivative 2 nonzero, $J \not\equiv_2 \Delta$. It is the negative neighbour LOCUS's theorem needed. A second one: for $F(Y) = Y^2 - 1$ over $\mathbb{Q}$, $J = -1$ has zero residual and a unit derivative but is not the jet of the separately selected root $+1$, so the shared constant term is a premise, not bookkeeping.
- **Quantifier scope in binomial inversion.** At a single index the forward row does not imply the inverse row: $n = 1$, $a_0 = a_1 = 0$, $b_0 = 1$, $b_1 = 0$ satisfies the forward equation at $n = 1$ while the inverse equation there would say $0 = -1$. A renderer that hides the uniform quantifier changes the theorem; CADENCE's and CONCORD's compact displays rely on the surrounding uniform quantification. It checked both directions in Lean.
- **A gap in a proposed contract.** VANTAGE's telescoping rule $\sum_{k=a}^{b}(G(k+1) - G(k)) = G(b+1) - G(a)$ needs $a \le b$ or an empty-range branch: at $a = 3$, $b = 1$, $G(k) = k$ the sum is empty and the right side is -1. Small, and exactly the kind of negative neighbour the suite is for; the VANTAGE card now records it.
- **Regular versus unit.** Cancelling a regular element from an equality and constructing its inverse need different theorems; the integer 2 is regular and not a unit. Section 9.2 now keeps M6's subcases apart.
- **The workspace allocation this review asked for** (question R3). For each need it names the existing Lean mechanism and the layer that would have to justify itself: local parameters are sections and the elaborator's context, with a readable dependency record as the addition; local facts are `have` declarations, with roles and source-to-evidence maps as the addition; conventions are instances and notation, with notices on meaning changes; routine search is tactics and simp sets, with named budgeted profiles and replay evidence; alternative views are definitions and transport theorems, with an object-indexed registry of accepted observations and routes; editing is elaborator snapshots, with transactional acceptance and migration receipts. The rule is to store state only when it enables an operation ordinary Lean does not expose, and to put this mapping into the baseline comparison so a registry cannot claim credit for what local contexts already do.
- **Entailment-based reuse of certificates.** A retained certificate may serve a new request when a checked adapter shows the request follows from its guarantee: the current context proves its guards, the object correspondence is established, and the retained relation entails the requested one under the current policy. Higher precision answers lower, a larger domain answers a smaller, a narrower interval answers a wider, and stronger assumptions or extra axioms do not carry over. This generalizes CONCORD's and CADENCE's weakening rules into one proof obligation per reuse, with exact identity as the fast path.
- **A visible-obligation boundary.** The default rendering must expose every unresolved condition and accepted distinction that affects domain, branch, precision, completeness, or trust policy, and may fold the routine proofs behind them. It is a rendering contract against the evidence graph, and it makes "what should remain visible" (S10) a checkable rule rather than a taste.
- **A blind semantic-reading test.** Before readers see the expanded evidence, ask them to reconstruct the guarantee of a compact result (object, domain, branch, precision, completeness, open conditions) against matched positive and negative neighbours with equally plausible formatting; then reveal the expansion and measure whether it repairs mistakes. This is the sharpest usability measure either synthesis proposes.
- **A toolchain-migration test** that changes the native-evaluation axiom scheme, with revalidation against the destination version's policy and a new receipt (question R7 made concrete), and a small merge experiment in which two branches independently change a route or a witness scope.
- **Ten questions of its own** for the next discussion, which overlap with R1–R10 and add two:

what an author may delegate by specification without a notice (Prism's rule, made a decision), and how much of a conditional result is visible by default, to be settled by the reading test.

10.4 Where the two still differ

The differences are of emphasis. It prefers eight broad commitments with source locators to a fine matrix, and is right that a fine matrix invites a score; this review keeps the fine rows because they are the backlog an implementation needs and marks their status. It treats this review's timings as historical observations and asks for a matched benchmark with balanced dense factors, sparse multivariate inputs, and large coefficients before any threshold is set; that is accepted, and the numbers here are now presented as a reason to prototype, not as a feasibility result. It orders the first slice as guarded algebra and complete solving first, then the formal-root jet; this review's order (certificate, cancellation, jet) is the same in substance. On the questions, it gives per-question assessments where this review gives modal answers; on the fourth-file rule, its sealed-holdout caveat is adopted above. Neither synthesis claims a measured usability result, and both say the next artifact should be a proved checker rather than a tenth design.

11 Recommendation

1. **Adopt the result-kind discipline as the specification of the computational boundary.** Table 6 is the contract; every request names a row, every producer returns a candidate for that row, and every promotion is a theorem application.
2. **Build the first three slices of Section 9.3 as a Lean library with retained records and no new syntax.** Ship them with the seventeen-row core suite and the positive neighbours from the reports' worked examples.
3. **Prove one checker sound end to end,** reification included, before adding a second checker or any grammar.
4. **Measure the strict lane on real multivariate certificates,** and set the threshold for the accelerated profile from that measurement.
5. **Write the `grobner` contract:** what it decides within its step and degree limits, over which coefficient domains, what it returns on failure, and whether it can emit a certificate for the strict lane.
6. **Test the bridge registry and the display:** omit one premise from each bridge and require failure; run the blind semantic-reading test on matched neighbours before choosing default visibility.
7. **Run the three-treatment comparison only after steps 2–4,** with the fourth-file rule for held-out tasks and with all infrastructure available to the Lean baseline. A library-only outcome is a success, as all nine reports say.

12 Questions for the third iteration

R1. How is a reflective checker proved sound in practice? The reports state the soundness theorem's type; none writes the denotation of a coefficient list into `Polynomial`, the evaluation homomorphism, or the treatment of opaque atoms in the reified syntax. The first prototype will answer this, and its answer decides how much of the verified core can be shared across checkers.

R2. Where does `grobner` sit? It is a bounded Gröbner-basis procedure that five reports name and none exercised. Is it the decision lane beside the certificate lane, does it emit a certificate, and what are its coefficient-domain, limit, and failure contracts?

R3. What is the accounting layer over Lean's own workspace? Which parts of the persistent workspace (Locus), transaction log (Accord), or anchor registry (Vantage) are new state,

and which are records of what `section`, `variable`, local instances, and `have` already do? The parallel synthesis's allocation (Section 10) is the proposed answer; the remaining question is its measured cost.

R4. Partial or total by default? MERIDIAN's partial-expression package and the total-operations-with-conditional-theorems camp both preserve meaning. Which lifts the existing corpus with less change, and which produces the more readable published document?

R5. Which package first, measured? S9 is the round's one open spread. A held-out test with the Bell file for EGF coordinates and the Abel file for formal series would settle it.

R6. When does the accelerated profile earn its axiom? At what certificate size, on which checker data, does `native_decide` beat kernel reduction? The threshold should be a number, not a policy.

R7. How do accepted certificates survive a toolchain upgrade? The native-evaluation axiom scheme changed at 4.29; the reports' pinned-environment rule handles a library upgrade but says nothing about a change in what an axiom inventory means.

R8. What does Leant contribute to the CAS examples specifically? Every report keeps Leant for structural composition; none identifies which obligations in the worked examples are compositions rather than single-lemma applications. The first three slices generate real obligations on which to measure this.

R9. Can the fifty-one mutations be executed? Each is a specification. Turning the seventeen-row core into Lean test files that the first slice must fail correctly is the cheapest way to make the next round's claims checkable.

R10. Is nine the right number? Nine reports produced one interface and about a dozen distinctive ideas. A third round of nine would likely produce the same interface again. Three reports, each required to prototype one slice in Lean and to open a new file, might produce more.

13 Conclusion

The second round did what the first round's syntheses asked. It took the consensus architecture as given, answered the twenty-two questions the same way nine times, and extended the design to computation by one idea that every report found independently: an answer is a value with a proof of a fixed specification, drawn from a small family of relations that cannot be exchanged for one another without a theorem. Around that idea the reports built a two-lane verified CAS whose strict lane is kernel-checked and whose accelerated lane discloses its axioms, a representation discipline that transports forward and reflects only by theorem, a precision calculus for formal series, a residual certificate that identifies computed jets with arbitrary solutions, and a provider boundary on which non-derivability is never a negation.

What the round did not do is compile anything. Running the toolchain shows that the design is closer to Lean than the reports assumed: every theorem the contracts would wrap exists, a bounded Gröbner tactic is already there, and a small strict certificate check costs about a second at degree 1280. The parallel synthesis, having reviewed this one, agrees on the next step and sharpens it. It is not a tenth design. It is one checker proved sound, with its denotation and its reification bridge, a library that retains what it checked, and the mutations that library must refuse, paired with the positive cases it must accept.

A Report cards

Each card gives the report's unit, its first slice, its most distinctive idea, its strongest worked example, and one weakness.

Locus. Unit: a workspace $(\mathcal{E}, \Gamma, \mathcal{O}, \mathcal{F}, \mathcal{V}, \mathcal{P})$ with only $\mathcal{E}; \Gamma$ deciding typing. First slice: the dyadic-germ residual jet from ProveIt's algebraic-germ file. Distinctive: the general residual-to-jet theorem with a unit hypothesis, Newton doubling as a second producer, four request purposes, the largest test matrix (28 rows). Example: $\Delta + 4\Delta^2 = 4Q/9$ with the Catalan closed form cross-check. Weakness: the workspace's guard-closure and rewrite-graph machinery is designed before the checker it would serve.

Accord. Unit: a transaction committing a result with its contract in a worksheet of objects, observations, and arguments. First slice: annotated reindexing plus a polynomial certificate. Distinctive: kind versus validation state; the fifth publication check on display strength; the two-square-root denesting theorem with degenerate cases inside it. Example: Abel binomial identity derived from $\exp((x+y)T) = \exp(xT)\exp(yT)$. Weakness: the grammar and state machine appendix restates the first round rather than the CAS.

Cadence. Unit: a transformation record with a declared guarantee, two-dimensional (semantic guarantee and evidence mechanism). First slice: $ax = b$ solved completely with set-valued output. Distinctive: guarded specialization; composition precision $\min(rN, M)$; the dual-number control; the `polyrith` rule; four suggestion statuses. Example: Lambert polynomials $P_{n+1} = (1+w)P_n' - (nw + 3n - 1)P_n$ through P_6. Weakness: 267 companion checks, but the derivative theorem is never connected to the analytic side.

Concord. Unit: a contracted query with demand-indexed observations. First slice: the Abel residual jet with finite reindexing and local derivative transfer. Distinctive: demand-driven precision with restriction as a weakening rule and precision in the cache key; the residual certificate proved by order improvement; the status vector; three negative questions. Example: the degree-four Abel coefficient $x(x-4a)^3/24$ by residual jet, with a trace whose every sentence is a checked node. Weakness: shortest report, and its Leant section adds nothing to the syntheses.

Facet. Unit: an object with exact presentations (data, validity, decode) and observations (relation only). First slice: a polynomial calculation with a guarded real-expression bridge and forged-native-output rejection. Distinctive: the $\mathbf{F}_2$ reflection failure; five nested arms L_0–L_4; contract-directed presentation planning; the insistence that "verified algorithms" means a verified core. Example: $P_{n+1} = (1+X^2)P_n'$ with $W^{(4)} = 16W + 40W^3 + 24W^5$. Weakness: its "modest relative guarantee" section is the substitution lemma the syntheses already said not to repeat.

Meridian. Unit: a computed object with a specification, whose properties form a partial order under proved implications. First slice: domain-preserving rational simplification with complete solving. Distinctive: the partial-expression model (D_E, v_E) with proved composition and specialization; $X \in (2X)$ over $\mathbb{Q}$ but not $\mathbb{Z}$; contract and guard surprise; `match_scalars`; `cbv`. Example: all derivatives of both real Lambert branches from one symbolic step, with the degree and leading-coefficient invariant. Weakness: the longest report, with a decisions table that restates the questions appendix.

Noema. Unit: a relation-typed calculation with a guard record. First slice: guarded cancellation with a polynomial checker and a stated strict path. Distinctive: the counting proof of the floor-square-root sum; the circularity rule; the rendered failure message; $D^n W = n! W^{n+1}$ as the minimal hybrid. Example: the root of $x^3 - x - 1$ bracketed in $[331/250, 53/40]$ with exact residuals. Weakness: its capability ladder's last rows are a research agenda presented beside an initial contract.

Prism. Unit: a representation $\mathrm{Rep}(s, r)$ anchored to an object; computations return typed claims. First slice: a polynomial certificate plus the Lambert derivative step. Distinctive: EGF coordinates with a proved inverse and binomial convolution; delegation by specification; three replay levels; the symbolic-parameter warning; the largest companion (2,475 checks including modular and dual-number controls). Example: the cross-multiplied Lambert certificate checked for $n \le 8$. Weakness: its question crosswalk groups questions rather than answering each.

Vantage. Unit: a certified change of representation for a specified observation, with exact presentations, restricted observations, and one-way abstractions kept apart. First slice: a normalized EGF coefficient proof. Distinctive: anchor-relative coherence; the Bell development with Touchard as polynomial algebra and Spivey by coefficient compiler; the Sturm chain; partial information without partial acceptance; the largest suite (26 rows) and companion (3,013 checks). Example: $\sqrt{3+2\sqrt{2}} = 1+\sqrt{2}$ by a balance certificate and by root isolation. Weakness: the longest report, its IR-node table and module table duplicate the syntheses, and its proposed telescoping contract omits the ordered-range premise (Section 10).

B The consolidated negative suite

All fifty-one canonical mutations, with the number of adversarial suites that contain each. Per-report identifiers are in `negative_suite.csv`.

Id	Mutation	Required behaviour	Suites
M1	Omit the inclusive endpoint of a finite reindexing	Coverage fails at the endpoint; the method is not silently replaced	8
M2	Use a non-injective index map as a bijection	Reject, or switch visibly to a multiplicity-aware theorem	2
M3	Apply the finite reindexing method to an infinite sum	Require the summability or rearrangement contract	2
M4	Replace neighbourhood equality by equality at one point	Derivative transfer stays open; no global smoothness is added	9
M5	Exchange a limit and an integral from pointwise convergence	The analytic theorem's hypotheses remain obligations	2
M6	Cancel a nonzero non-unit, or use a power certificate, with zero divisors	Require unit or regularity evidence	5
M7	Cancel a denominator that can vanish, or evaluate at the excluded point	Keep the guard, split cases, or return a piecewise result	8
M8	Forget the domain of a partial function after normalizing it	Extension is a new object with its own agreement theorem	2
M9	Return the negative radical candidate with the correct square	Reject the principal-root specification	8
M10	Return the formal root with the wrong constant coefficient	Reject the branch condition of the root specification	1
M11	Request a coefficient beyond the certified precision of a jet	Expose the failed index bound; recompute or refine	3
M12	Differentiate a jet and keep its input precision	Lower the output order or request one more coefficient	7
M13	Read a formal identity or jet as an analytic statement	Require a convergence, realization, or remainder theorem	6
M14	Promote finitely many agreeing coefficients to a universal claim	Retain only the finite statement	5
M15	Present verified roots as the complete set, or omit a root	Soundness may pass; completeness must fail	9
M16	Reuse a branch-local fact in a sibling branch through a cache	Reject the context embedding	6
M17	Deliver a valid result after undo or an origin change	Mark it stale; it cannot mutate the current state	8
M18	Change an instance or definition with the printed statement unchanged	Invalidate the seal and display the semantic change	4
M19	Import a second package with a conflicting convention or name	Explicit selection; no silent shadowing	2
M20	Let proof search instantiate a meaning-bearing statement hole	Keep the statement unresolved	1
M21	Display suggested tactic text that differs from the probed tactic	Replay the exact displayed edit before calling it verified	8

Id	Mutation	Required behaviour	Suites
M22	Treat a reduction in visible goals as theorem completion	Report local progress only	2
M23	Admit an unapproved axiom transitively, or forge an evidence tag	Fail the publication policy	5
M24	Accept a verified algorithm's native output without checked execution	The algorithm theorem alone is insufficient	2
M25	Insert a false but unused intermediate assertion	Reject the document	8
M26	Add a constructor to a datatype handled by a structural plan	Recompute coverage	1
M27	Present exhaustion, a timeout, or a missing status field as a refutation	Return a non-logical outcome	8
M28	Supply a finite behavioral test for a universal specification	Retain the finite fact; the universal obligation stays open	1
M29	Introduce a competing representation route	Preserve the selected route, request a choice, or apply coherence	3
M30	Reflect an equality back along a non-injective map	Require reflection evidence	3
M31	Retrieve the benchmark target through an alias	The run is invalid	2
M32	Commit incompatible shared data assignments	Reject the transaction and restore the origin	3
M33	Transport natural subtraction or division without its guard	Expose the order or divisibility obligation	5
M34	Corrupt one coefficient of a certificate	The checker rejects it regardless of the producer	4
M35	Strengthen a group or rational algebra to a field	A statement change, never a proof improvement	4
M36	Replace an arbitrary solution by the canonical construction	Require an equality or uniqueness bridge	3
M37	Feed a noncommutative expression to a commutative normalizer	Reject the missing capability	1
M38	Display an enclosure as exact, or infer a sign from an interval containing zero	Exact bounds are stored; zero-containing intervals are inconclusive	5
M39	Solve a parameterized equation without its singular branch	Report a conditional answer or require coverage	3
M40	Label multiplied-back factors as irreducible	Irreducibility needs separate evidence	1
M41	Use a rational-coefficient membership witness for an integer ideal	Reject the coefficient-domain mismatch	1
M42	Treat an acknowledged notice as discharging a guard	A notice never discharges an obligation	1
M43	Use a result to establish the guard that justifies it	Reject cyclic assembly	1
M44	Bind a provider's expression to the wrong origin by name	Reject the typed origin binding	1
M45	Remove an analytic hypothesis, or use a rational form at its pole	Expose the requirement; the header must not strengthen itself	2
M46	Substitute a series with nonzero constant into the simple composition rule	Generate the admissibility obligation	2
M47	Use a conditional result unconditionally, or swap dependent quantifiers	Reject the scope or dependency change	1
M48	Force an already-short corollary into a longer narrative	New syntax must not add declarations	1
M49	Multiply a strict inequality by a merely nonnegative factor	Require strict positivity or handle zero	1
M50	Infer equality of algebraic numbers from a shared polynomial	Require matching root identity	1

Id	Mutation	Required behaviour	Suites
M51	Specialize a guarded computation where the guard fails	Reinstantiate the guard and locate its origin	1

C Experiments and reproducibility

The Lean experiments were run on 5 September 2026 with `lake env lean <file>` from the ProveIt checkout (toolchain `leanprover/lean4:v4.32.0`, Mathlib package revision `81a5d257c8`, Windows 11, x86_64). The files are in `experiments/lean/` with their verbatim compiler logs:

- `Axioms.lean`: the `native_decide` axiom probe, with its log.
- `PolyCert.lean`: the unverified dense-polynomial certificate checker, kernel and native timings for degrees 10 to 1280 (two runs on 6 September 2026, `PolyCert.run1.log` and `PolyCert.run2.log`, which replace the 5 September runs after the parallel synthesis noted that the shipped file had lost its corrupted-certificate case), the corrupted certificate proved rejected, the Catalan residual jet, and the derivative precision loss.
- `LinComb.lean`: the same identity family through `linear_combination`.
- `Supply.lean`: the `#check` sweep of Table 4, the `Polynomial` computability probe, and the `grobner` and `polyrith` probes.

Timings are from the Lean profiler (`set_option profiler true`) and vary between runs with machine load; both runs are shipped. The checker is unproved: the experiments measure cost and axiom inventories, not soundness. The parallel synthesis's independent reruns of the axiom probe, with and without a Mathlib import, and its `grobner` and `linear_combination` checks are under `docs/round-2/synthesis/evidence/secondary-review/lean/`.

The data files `feature_matrix.csv`, `negative_suite.csv`, and `question_tally.csv` are generated by `tools/make_tables.py`, which also prints the LaTeX bodies of Tables 2 and 7. The report builds with `latexmk -pdf unified_report.tex` or the shipped `build.sh`.

D Source map

All nine reports are under `docs/round-2/ideas/` in the Brainstorm repository at the revision this review was written against; the directory names are `Locus_Proof_Language_Proposal`, `accord`, `cadence`, `concord`, `facet`, `meridian`, `noema`, `prism`, and `vantage`, each containing the TeX source, the built PDF, a README, a Python companion with its recorded results, and a source register. The reports pin the Brainstorm revision they read (`494be9d5`), the ProveIt revision they inspected (`24ce8bd7` for seven reports, Git blob identities for Locus, and `7c4e3f10` for Facet), and the Leant revision (`3a40904b`). The two first-round syntheses are under `docs/round-1/synthesis/` and `docs/round-1/unified_report/`; the parallel second-round synthesis, with its convergence register, review memos, and Lean evidence, is under `docs/round-2/synthesis/`.

References

[1] *Locus: Mathematical Workspaces and Certified Computation.* Round-2 report, `docs/round-2/ideas/Locus_Proof_Language_Proposal/locus.tex`, 5 September 2026.

[2] *Accord: A Certified Worksheet Language for Mathematical Reasoning and Symbolic Computation.* `docs/round-2/ideas/accord/accord.tex`.

[3] *Cadence: A Mathematical Language of Certified Transformations.* `docs/round-2/ideas/cadence/cadence.tex`.

[4] *Concord: A Query-Directed Proof Language with a Verified Symbolic Core.* `docs/round-2/ideas/concord/concord.tex`.

[5] *Facet: Mathematical Objects with Certified Computational Presentations.* `docs/round-2/ideas/facet/facet.tex`.

[6] *Meridian: A Mathematical Language with Certified Computation.* `docs/round-2/ideas/meridian/meridian.tex`.

[7] *Noema: Mathematical Objects, Scoped Relations, and Certified Computation.* `docs/round-2/ideas/noema/noema.tex`.

[8] *Prism: A Proof Language with Certified Mathematical Computation.* `docs/round-2/ideas/prism/prism.tex`.

[9] *Vantage: Mathematical Objects, Explicit Views, and Certified Computation.* `docs/round-2/ideas/vantage/vantage.tex`.

[10] Codex. *Mathematical Ideas, Checked Evidence: A Unified Evaluation of Nine Proof-Language Proposals.* Round-1 synthesis, `docs/round-1/synthesis/unified-report.tex`.

[11] Claude (Anthropic). *Nine Blueprints for a Mathematician's Proof Language over Lean.* Round-1 synthesis, `docs/round-1/unified_report/unified_report.tex`.

[12] Codex. *Mathematical Objects, Certified Computation: A Unified Evaluation of Nine Proof-Language Proposals.* Parallel round-2 synthesis, revised 6 September 2026 after reviewing this document; `docs/round-2/synthesis/unified-report.tex`, with review memos under `evidence/secondary-review/`.

[13] V. Reshetnikov and contributors. ProveIt. https://github.com/VladimirReshetnikov/ProveIt. Files cited by the reports: `Analysis/FabiusFunction/Lean/FabiusFunction/BinomialInversion.lean`, `AutonomousIteratedDeriv.lean`, `AbelPolynomialSeries.lean`, `AlgebraicInverseGerm.lean`, `LambertWHigherDerivatives.lean`, `BellShiftEGF.lean`, `BellGeneratingFunctions.lean`, and `NumberTheory/IntegerSums/Lean/IntegerSums/FloorSqrtSum.lean`.

[14] V. Reshetnikov and contributors. Leant. https://github.com/VladimirReshetnikov/Leant. Files cited by the reports: `src/Leant/Synth/Verification.hs`, `src/Leant/Synth/Engine.hs`, `src/Leant/Synth/Fragment.hs`, `src/Main.hs`, `docs/synth-internals.md`.

[15] Lean team. *Validating a Lean Proof.* Lean Language Reference. https://lean-lang.org/doc/reference/latest/ValidatingProofs/.

[16] Lean team. *Axioms*, including native evaluation and `native_decide`. Lean Language Reference. https://lean-lang.org/doc/reference/latest/Axioms/.

[17] Mathlib contributors. *Mathlib.Tactic.Polyrith.* https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Polyrith.html.

[18] Mathlib contributors. *Mathlib.Tactic.LinearCombination.* https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/LinearCombination.html.

[19] W. Li, G. O. Passmore, L. C. Paulson. Deciding univariate polynomial problems using untrusted certificates in Isabelle/HOL. *J. Automated Reasoning*, 2017. Cited by Locus, Concord, Vantage.

[20] H. Shen, J. Guo, J. Liu, L. Zhi. Automated tactics for polynomial reasoning in Lean 4. arXiv:2604.13514, April 2026. Cited by Locus, Meridian.

[21] M. Dénès, A. Mörtberg, V. Siles. A refinement-based approach to computational algebra in Coq. Cited by Accord, Facet, Vantage.

[22] J. Harrison, L. Théry. A sceptic's approach to combining HOL and Maple. *J. Automated Reasoning* 21 (1998). Cited by Facet, Noema, Prism.

[23] J. H. Davenport. Towards verified polynomial factorisation. arXiv:2409.09533, 2024. Cited by Meridian.

[24] G. Melquiond and contributors. Interval package for Rocq. https://coqinterval.gitlabpages.inria.fr/. Cited by Cadence, Facet, Meridian, Noema, Prism.

[25] R. Thiemann, A. Yamada, S. J. C. Joosten. Algebraic numbers in Isabelle/HOL. Archive of Formal Proofs. Cited by Accord, Concord, Facet.

[26] R. Y. Lewis, M. Wu. A bi-directional extensible interface between Lean and Mathematica. *J. Automated Reasoning* 66 (2022). Cited by Noema.

[27] P. Massot. Teaching mathematics using Lean and controlled natural language. ITP 2024. Cited by Locus, Concord, Noema.